Identifying Slow Examples

Throughout this book, we're going to give you advice on how to keep your specs running quickly. To understand where the biggest bottlenecks are in your suite, you need to be able to identify the slowest examples.

RSpec's spec runner can help you do so. Consider the following group of examples that take too long to run:

02-running-specs/03/spec/slow_spec.rb

```
RSpec.describe 'The sleep() method' do
  it('can sleep for 0.1 second') { sleep 0.1 }
  it('can sleep for 0.2 second') { sleep 0.2 }
  it('can sleep for 0.3 second') { sleep 0.3 }
  it('can sleep for 0.4 second') { sleep 0.4 }
  it('can sleep for 0.5 second') { sleep 0.5 }
end
```

We can ask RSpec to list the top time-wasters by passing the --profile option along with the number of offenders we'd like to see:

```
$ rspec --profile 2
.....

Top 2 slowest examples (0.90618 seconds, 59.9% of total time):
  The sleep() method can sleep for 0.5 second
    0.50118 seconds ./spec/slow_spec.rb:6
  The sleep() method can sleep for 0.4 second
    0.40501 seconds ./spec/slow_spec.rb:5

Finished in 1.51 seconds (files took 0.08911 seconds to load)
5 examples, 0 failures
```

Just two examples are taking over half our test time. Better get optimizing!

Running Just What You Need

In the examples in this chapter, we've always run all the specs together. On a real project, you don't necessarily want to load your entire test suite every time you invoke RSpec.

If you're diagnosing a specific failure, for instance, you'll want to run just that one example. If you're trying to get rapid feedback on your design, you can bypass slow or unrelated specs.

The easiest way to narrow down your test run is to pass a list of file or directory names to rspec:

```
$ rspec spec/unit                  # Load *_spec.rb in this dir and subdirs
$ rspec spec/unit/specific_spec.rb # Load just one spec file
```